Student Task Manager Python Application Report

Concept Based Assignment

Name: om supe Roll number: 11

Course: cse aiml Date: 28 October 2026

Project overview

Student Task Manager is a Python command-line application that records assignments and study tasks. It lets a student add a task, view the task list, mark work complete, delete a task, and check overall progress. Tasks remain available after the program closes because the application saves them in a local JSON file.

This project demonstrates how basic Python concepts can work together in a complete application. The program uses functions to separate responsibilities, dictionaries to describe individual tasks, and a list to hold the collection. Validation and exception handling help the application respond clearly when the user enters an invalid value or the data file cannot be read.

Problem and objectives

A student may need to track several small pieces of work at the same time. A simple task list provides one place to record what needs to be done and whether it is complete. The application keeps the scope small enough to understand, run, and explain during an assignment demonstration.

The objectives are to create tasks with useful details, preserve records between runs, update completion status using stable task IDs, calculate progress, and handle common errors. The project also includes repeatable tests and documentation so another person can run it without installing extra Python packages.

Requirements and scope

The application requires Python 3.10 or newer and a terminal. It was tested on Windows with Python 3.12.14. All imports belong to the Python standard library. The interface uses argparse for command parsing and generated help messages [1].

Command	Purpose	Main input
add	Create a task	Title, optional due date and priority
list	Display tasks	Optional status filter
complete	Mark work complete	Existing task ID
delete	Remove a task	Existing task ID
summary	Show overall progress	No additional input

The application is designed for one local user. It stores and displays due dates, but it does not send reminders or schedule tasks automatically. Priority is a label; it does not change the display order.

Design and application workflow

Each command parses arguments, loads and validates the data, performs an operation, and displays a result. Changes are saved before success is printed. Listing and summary commands only read the file. Functions such as add_task and summarize handle task operations, while main connects them to the command interface. This separation makes the operations easier to test.

Data representation

The file contains a next_id counter and a tasks list. Each task has an integer ID, title, due date string, priority string, and Boolean completed value. An empty date means no date was supplied. JSON stores this structure as readable text [2].

```
{
  "next_id": 2,
  "tasks": [{"id": 1, "title": "Finish Python report",
             "due": "2026-11-05", "priority": "high",
             "completed": false}]
}
```

The counter increases when a task is added. Deleted IDs are not reused. The loader rejects duplicate IDs and a counter that is smaller than or equal to an existing ID.

Python concepts applied

Concept	Application in the project
Lists and dictionaries	Store the task collection and named task fields
Functions	Separate validation, storage, and task operations
Conditions and iteration	Route commands, find IDs, and filter tasks
Boolean values and arithmetic	Count completed tasks and calculate progress
File handling and modules	Read and write JSON using pathlib and json
Exceptions	Report invalid values and file errors clearly

Progress is completed tasks divided by total tasks, multiplied by 100. An empty list returns 0.0 percent to avoid division by zero. Lookup and filtering scan the list.

Validation and storage behavior

Titles must contain text, fit within 120 characters, and remain on one line. Priorities must be low, medium, or high. A due date must be a real calendar date written as YYYY-MM-DD. A missing task ID produces a clear error. Invalid stored data stops the command before any update is saved.

Saving writes JSON to a temporary file in the same directory and then replaces the existing file. This reduces the chance of leaving a partially written main file. It is not a backup system and does not provide coordination between simultaneous processes.

Execution and observed output

The application was executed with a separate temporary data file, using a new Python process for each command. Two tasks were added, the list was displayed, task 1 was completed, and a summary was requested. The output below was captured from that execution. The --data option selected the temporary file and is omitted here for readability.

```
> python task_manager.py list
  1 | pending | high   | 2026-11-05 | Finish Python report
  2 | pending | medium | no date    | Revise functions

> python task_manager.py complete 1
Completed task 1: Finish Python report

> python task_manager.py summary
Total: 2 | Completed: 1 | Pending: 1 | Progress: 50.0%

> python task_manager.py list --status pending
  2 | pending | medium | no date    | Revise functions

> python task_manager.py delete 2
Deleted task 2: Revise functions
```

The summary correctly reports one completed task out of two. The pending filter returns only task 2. Since each command started a separate process, the demonstration also confirms that the task records and completion status were loaded from disk between commands.

Testing and results

The test suite uses unittest and temporary directories [3]. It runs with the command python -m unittest -v from the project folder. All 16 tests passed under Python 3.12.14. The table groups the checks by behavior; the counts refer to test methods, some of which check several inputs.

Test area	Tests	Result
Empty storage and save then reload	2	Passed
Title, date, and priority validation	4	Passed
Completion, filtering, deletion, and missing IDs	4	Passed
Empty and nonempty progress calculations	1	Passed
Malformed JSON, invalid records, and duplicate IDs	3	Passed
Simulated write failure preserves the old file	1	Passed
Full command workflow and persistence	1	Passed
Total	16	Passed

An additional process-based check supplied the impossible date 2026-02-30. The command exited with error status 1 and left the stored file unchanged. The tests support the listed behaviors; they do not establish performance at large scale or compatibility with every operating system.

Installation and demonstration steps

Download or extract the project folder and open a terminal inside it. Confirm that Python is installed, then run the commands below. On systems that use python3 as the interpreter name, substitute python3 for python.

```
python --version
python task_manager.py --help
python task_manager.py add "Finish Python report" --due 2026-11-05
python task_manager.py list
python task_manager.py complete 1
python task_manager.py summary
python -m unittest -v
```

The example assumes a new default data file, where the first task receives ID 1. For an existing file, use the ID shown by list. The first successful change creates task_data.json beside the script. A different location can be supplied with --data before the command. The default data file is excluded from Git so personal tasks are not included in the repository.

Repository contents

The project contains task_manager.py, test_task_manager.py, README.md, .gitignore, and the docs folder. The report is supplied as a PDF for submission and as Markdown for editing. The README explains the commands, concepts, tests, and limitations.

The code, tests, and completed report are available in the public GitHub repository at https://github.com/omsupe28/student-task-manager. The repository can be cloned or downloaded to run the application and review its tests. This link can accompany the assignment submission.

Limitations and possible improvements

The program supports a single local user and has no graphical interface, authentication, cloud synchronization, reminders, or undo for deleted tasks. Concurrent writes by separate processes can lose changes. The JSON file is readable by anyone with access to its folder, so it should not contain sensitive information.

Useful future improvements include a task editing command, optional sorting by due date or priority, and an overdue filter. A graphical interface could make the program easier for users who do not work in a terminal. These additions would require their own validation and tests.

Conclusion

Student Task Manager combines core Python concepts in a working application with persistent storage and a repeatable test suite. The recorded demonstration shows that tasks can be created, completed, filtered, and deleted across separate runs. The 16 passing tests provide evidence for the core operations and common error cases. The project and report are prepared for review and submission.

References

1. Python Software Foundation. argparse documentation. https://docs.python.org/3/library/argparse.html

2. Python Software Foundation. json documentation. https://docs.python.org/3/library/json.html

3. Python Software Foundation. unittest documentation. https://docs.python.org/3/library/unittest.html

Documentation accessed 28 October 2026.